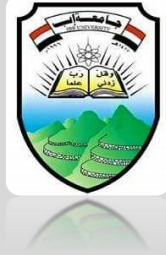


الجمهورية اليمنية

جامعة إب



كلية الحاسبات

علوم الحاسوب وتقنية المعلومات

المستوى الرابع

مقرر: معالجة صور رقمية - عملي

DIP-Lab-6

اسم الطالب: رؤوف صادق عبده محمد قايد العقاب.

المجموعة: (A1).

بإشراف من مدرس المقرر: م/ مالك المصنف

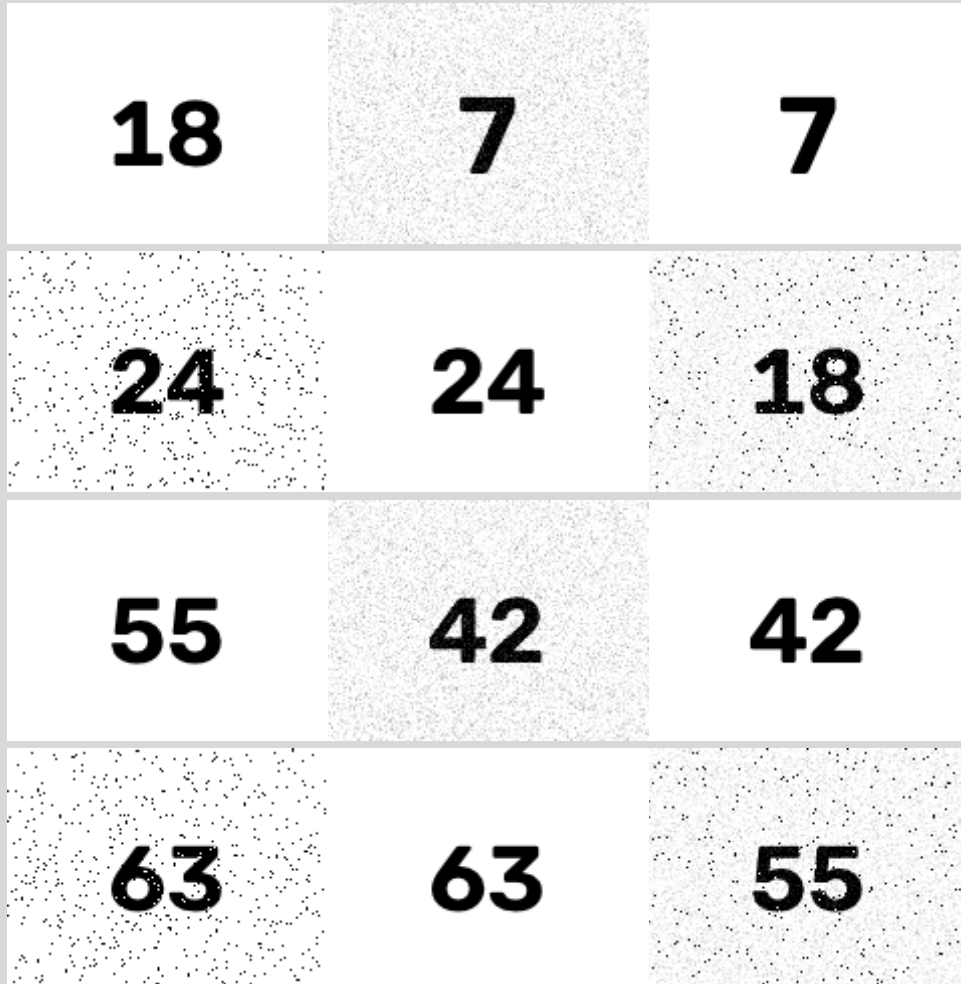
1448

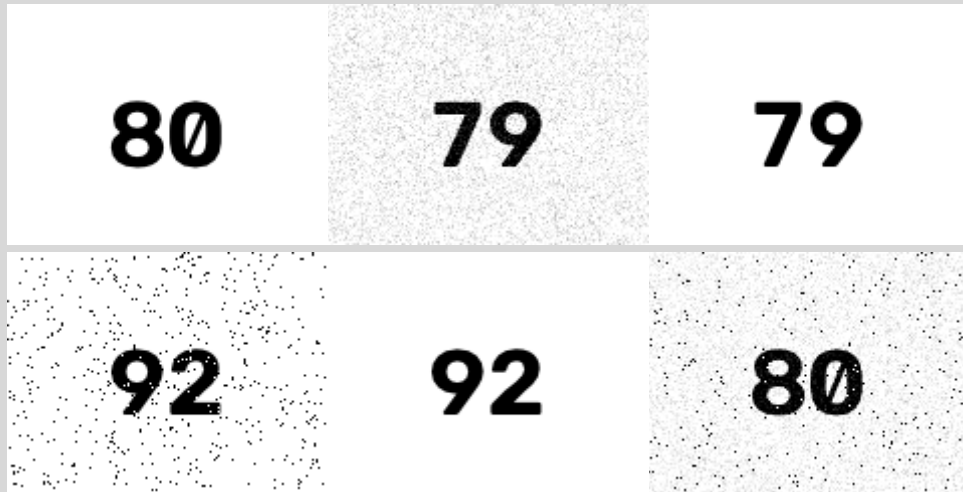
2027/2026

1. الفلاتر وتقنيات معالجة الصور المستخدمة:

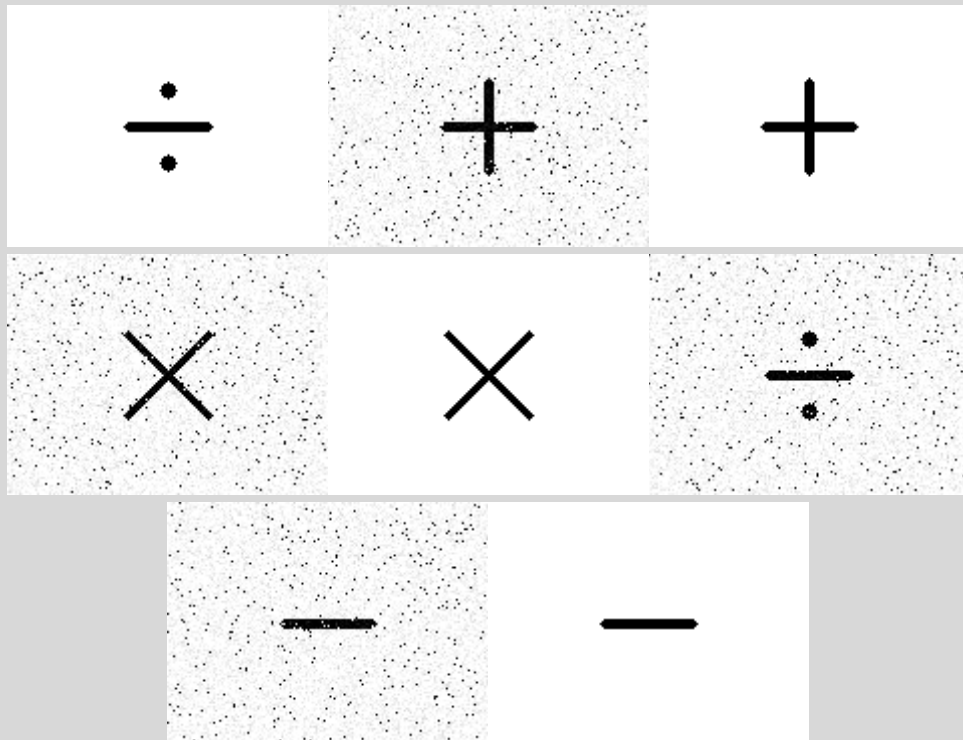
1. **Grayscale**: تحويل الصورة إلى تدرج رمادي .
2. **Median Filter**: إزالة الضوضاء، خصوصًا Salt & Pepper.
3. **Gaussian Filter**: تنعيم الصورة وتقليل الضوضاء .
4. **CLAHE**: تحسين التباين وإبراز الأرقام والرموز .
5. **Otsu Thresholding**: تحويل الصورة إلى Binary تلقائيًا .
6. **Adaptive Thresholding**: معالجة الصور ذات الإضاءة غير المتجانسة عند الحاجة .
7. **Morphological Opening**: إزالة النقاط والضوضاء الصغيرة .
8. **Morphological Closing**: سد الفجوات داخل الأرقام والرموز .
9. **Canny Edge Detection**: استخراج حواف الأرقام والرموز .
10. **Connected Components / Contours**: تحديد المكونات وفصل الأرقام والرموز.

2. صور الأرقام التي تم استخدامها:





3. صور العمليات:



4. خوارزمية عمل البرنامج بالضبط

1. إدخال 3 صور:

- صورة الرقم الأول .
 - صورة العملية الحسابية .
 - صورة الرقم الثاني .
2. تحويل كل صورة إلى Grayscale .

3. إزالة الضوضاء :

- يبدأ البرنامج بالـ **Median Filter** .
- وإذا كانت النتيجة غير مناسبة يستخدم Gaussian ضمن Pipeline بديل .

4. تحسين الصورة عند الحاجة :

- استخدام **CLAHE** لتحسين التباين في الصور الصعبة .

5. تحويل الصورة إلى Binary :

- استخدام **Otsu Thresholding** .
- استخدام **Adaptive Thresholding** كبديل عند الحاجة .

6. تطبيق العمليات Morphology :

- Opening لإزالة الأجزاء الصغيرة غير المرغوبة .
- Closing لسد الفجوات داخل الأرقام والرموز .

7. استخراج الحواف :

- تطبيق **Canny Edge Detection** للحصول على Edge Map .

8. اكتشاف المكونات :

- استخدام **Contours / Connected Components** لاكتشاف الأجزاء الموجودة في الصورة .
- إزالة المكونات الصغيرة الناتجة عن الضوضاء .

9. تجزئة الرقم :

- إذا كان الرقم مكوناً من أكثر من خانة، يتم تحديد Bounding Box لكل خانة .
- ترتيب الخانات من اليسار إلى اليمين .
- قص كل خانة منفردة .

10. توحيد حجم الخانات :

- تحويل كل رقم إلى حجم قياسي .
- الحفاظ على Aspect Ratio .
- توسيط الرقم داخل الصورة .

11. التعرف على الأرقام :

- مقارنة كل خانة مع Templates للأرقام من 0 إلى 9 باستخدام **Template Matching** .
- اختيار الرقم صاحب أعلى درجة تطابق .
- تجميع الخانات لتكوين الرقم الكامل، مثل:

$$92 \rightarrow 9 + 2$$

12. التعرف على العملية الحسابية :

- تحليل شكل المكونات .
- استخدام **Template Matching** للتأكيد .
- التعرف على:

$$+ , - , \times , \div$$

- بالنسبة لـ $\div$ يتم التعامل مع مكوناته الثلاثة: نقطة علوية + خط + نقطة سفلية .
- 13. **Fallback عند ضعف النتيجة :**
 - إذا كانت درجة الثقة منخفضة، يعيد البرنامج معالجة الصورة باستخدام Pipeline بديل .
 - يتم اختيار أفضل نتيجة .
- 14. **بناء التعبير الحسابي:**
 - بعد التعرف على الصور الثلاث:
 - الرقم الأول + العملية + الرقم الثاني
- 15. **التحقق من التعبير :**
 - التأكد من أن الرقمين والعملية تم التعرف عليهم بشكل صحيح .
 - منع القسمة على صفر .
- 16. **تنفيذ العملية**
 - استخدام دوال حسابية صريحة مثل الجمع والطرح والضرب والقسمة .
 - لا يتم استخدام `eval()` .
- 17. **عرض النتيجة في واجهة Tkinter :**
 - التعبير المتعرف عليه .
 - العملية الحسابية .
 - النتيجة النهائية .

5. الكود البرمجي لوحدة النظام (Source Code Screenshots)

أولاً: كود المعالجة المسبقة وخطوط الأنايبب (`preprocessing.py`):

```
import cv2
import numpy as np

def to_grayscale(image: np.ndarray) -> np.ndarray:
    """Converts image to grayscale if not already."""
    if len(image.shape) == 3 and image.shape[2] == 3:
        return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    elif len(image.shape) == 3 and image.shape[2] == 4:
        return cv2.cvtColor(image, cv2.COLOR_BGRA2GRAY)
    return image.copy()

def apply_gaussian_blur(gray: np.ndarray, ksize=(3, 3), sigma=0) -> np.ndarray:
    """Applies Gaussian smoothing to suppress Gaussian noise."""
    if ksize[0] % 2 == 0 or ksize[1] % 2 == 0:
        ksize = (ksize[0] | 1, ksize[1] | 1)
    return cv2.GaussianBlur(gray, ksize, sigma)

def apply_median_filter(gray: np.ndarray, ksize=3) -> np.ndarray:
    """Applies Median filtering - highly effective against Salt & Pepper noise."""
    if ksize % 2 == 0:
        ksize += 1
    return cv2.medianBlur(gray, ksize)

def apply_clahe(gray: np.ndarray, clip_limit=2.0, tile_grid_size=(8, 8)) -> np.ndarray:
    """Contrast Limited Adaptive Histogram Equalization for contrast enhancement."""
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_grid_size)
```

```

        return clahe.apply(gray)

def apply_threshold(gray: np.ndarray, method="otsu", invert=True) -> np.ndarray:
    """
    Thresholds the image to return a binary image where foreground (digits/operator) is 255
    (white)
    and background is 0 (black).
    """
    thresh_type = cv2.THRESH_BINARY_INV if invert else cv2.THRESH_BINARY

    if method == "otsu":
        _, binary = cv2.threshold(gray, 0, 255, thresh_type + cv2.THRESH_OTSU)
        return binary
    elif method == "adaptive":
        binary = cv2.adaptiveThreshold(
            gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, thresh_type, 15, 4
        )
        return binary
    else: # simple fixed threshold
        _, binary = cv2.threshold(gray, 127, 255, thresh_type)
        return binary

def apply_morphology(binary: np.ndarray, op="open", ksize=3) -> np.ndarray:
    """
    Applies morphological operations:
    - 'open': erosion followed by dilation (removes small noise specks).
    - 'close': dilation followed by erosion (bridges small gaps in strokes).
    """
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (ksize, ksize))
    if op == "open":
        return cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
    elif op == "close":
        return cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)
    return binary

def run_pipeline(image: np.ndarray, pipeline_name="pipeline_1") -> dict:
    """
    Executes a specific preprocessing pipeline and returns intermediate images at each step
    for visualization and subsequent edge detection/segmentation.
    Output binary has foreground = 255, background = 0.
    """
    gray = to_grayscale(image)

    if pipeline_name == "pipeline_1":
        # Pipeline 1: Median filter + Otsu + Morph Close
        # Ideal for Salt & Pepper noise and standard text
        denoised = apply_median_filter(gray, ksize=3)
        binary = apply_threshold(denoised, method="otsu", invert=True)
        morph = apply_morphology(binary, op="close", ksize=3)

    elif pipeline_name == "pipeline_2":
        # Pipeline 2: Gaussian Blur + Otsu + Morph Open + Close
        # Ideal for Gaussian noise and slight stroke irregularities
        denoised = apply_gaussian_blur(gray, ksize=(3, 3), sigma=0.8)
        binary = apply_threshold(denoised, method="otsu", invert=True)
        opened = apply_morphology(binary, op="open", ksize=3)
        morph = apply_morphology(opened, op="close", ksize=3)

    elif pipeline_name == "pipeline_3":
        # Pipeline 3: Median filter + CLAHE + Otsu + Morph Close
        # Ideal for combined noise or lower contrast images
        denoised = apply_median_filter(gray, ksize=3)
        enhanced = apply_clahe(denoised, clip_limit=2.0)
        binary = apply_threshold(enhanced, method="otsu", invert=True)
        morph = apply_morphology(binary, op="close", ksize=3)

    else:
        # Default fallback
        denoised = apply_median_filter(gray, ksize=3)
        binary = apply_threshold(denoised, method="otsu", invert=True)
        morph = binary

    return {
        "pipeline_name": pipeline_name,

```

```

        "original": image,
        "gray": gray,
        "denoised": denoised,
        "binary": binary,
        "morph": morph
    }

```

ثانياً: كود كشف الحواف والمكونات المتصلة (edge_detection.py):

```

"""
edge_detection.py - Edge detection and contour/component extraction module.
Applies Canny edge detection with dynamic thresholds, extracts clean contours,
and provides connected components analysis (vital for multi-element operators like '÷').
"""

import cv2
import numpy as np

def apply_canny(image: np.ndarray, low_threshold=None, high_threshold=None) -> np.ndarray:
    """
    Applies Canny edge detector with automatic thresholding if not provided.
    Accepts grayscale or smoothed image.
    """
    if len(image.shape) == 3:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    if low_threshold is None or high_threshold is None:
        # Dynamic Otsu-based thresholding
        otsu_val, _ = cv2.threshold(image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        high_threshold = otsu_val
        low_threshold = otsu_val * 0.5

    edges = cv2.Canny(image, int(low_threshold), int(high_threshold))
    return edges

def find_clean_contours(binary_mask: np.ndarray, min_area=25) -> List[dict]:
    """
    Finds external contours and filters out noise contours with area < min_area.
    Returns list of contour info dicts with contour, bounding box (x, y, w, h), and area.
    """
    contours, hierarchy = cv2.findContours(
        binary_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    clean = []
    for c in contours:
        area = cv2.contourArea(c)
        if area >= min_area:
            x, y, w, h = cv2.boundingRect(c)
            clean.append({
                "contour": c,
                "bbox": (x, y, w, h),
                "area": area,
                "aspect_ratio": float(w) / max(1, h)
            })

    return clean

def find_connected_components(binary_mask: np.ndarray, min_area=20) -> List[dict]:
    """
    Extracts connected components from a binary image (where foreground is 255).
    Crucial for distinguishing operations like '÷' (which has 3 distinct components: dot, bar, dot).
    Returns list of component dicts filtered by min_area.
    """
    num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
        binary_mask, connectivity=8
    )

    components = []

```

```

# Label 0 is background, so iterate from 1 to num_labels - 1
for label in range(1, num_labels):
    area = stats[label, cv2.CC_STAT_AREA]
    if area >= min_area:
        x = stats[label, cv2.CC_STAT_LEFT]
        y = stats[label, cv2.CC_STAT_TOP]
        w = stats[label, cv2.CC_STAT_WIDTH]
        h = stats[label, cv2.CC_STAT_HEIGHT]
        cx, cy = centroids[label]

        # Mask for this specific component
        comp_mask = np.uint8(labels == label) * 255
        crop = comp_mask[y:y+h, x:x+w]

        components.append({
            "Label": label,
            "bbox": (x, y, w, h),
            "area": area,
            "centroid": (float(cx), float(cy)),
            "aspect_ratio": float(w) / max(1, h),
            "mask": comp_mask,
            "crop": crop
        })

    return components

def clean_edges_with_components(edges: np.ndarray, binary_mask: np.ndarray, min_area=25) ->
np.ndarray:
    """
    Cleans edge image by suppressing edges that do not belong to significant connected
    components.
    Eliminates stray noise edges caused by salt & pepper noise.
    """
    comps = find_connected_components(binary_mask, min_area=min_area)
    valid_mask = np.zeros_like(binary_mask)
    for comp in comps:
        valid_mask = cv2.bitwise_or(valid_mask, comp["mask"])

    # Dilate valid mask slightly to cover edge boundaries
    dilated_mask = cv2.dilate(valid_mask, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))
    clean_edges = cv2.bitwise_and(edges, dilated_mask)
    return clean_edges

```

ثالثاً: كود التجزئة وتوحيد المقاس المعياري (segmentation.py):

```

"""
segmentation.py - Segmentation module for extracting individual digits.
Sorts candidate bounding boxes Left -> Right, isolates each digit ROI,
and applies standardized normalization matching the template generator.
"""

import cv2
import numpy as np

DEFAULT_TARGET_SIZE = (32, 32)

def normalize_roi(roi_binary: np.ndarray, target_size=DEFAULT_TARGET_SIZE) -> np.ndarray:
    """
    Standardizes a segmented digit binary mask (255=foreground, 0=background)
    into target_size with aspect-ratio preserved centering.
    Guarantees 100% consistency with generate_templates.py.
    """
    th, tw = target_size
    pad_box = np.zeros((th, tw), dtype=np.uint8)

    # Locate exact non-zero boundaries
    coords = cv2.findNonZero(roi_binary)
    if coords is None:
        return pad_box

```

```

x, y, w, h = cv2.boundingRect(coords)
if w <= 0 or h <= 0:
    return pad_box

crop = roi_binary[y:y+h, x:x+w]

# Scale to fit inside (target - 4) with preserved aspect ratio
max_dim = max(w, h)
margin = 4
scale = (min(th, tw) - margin) / float(max_dim)
new_w = max(1, int(w * scale))
new_h = max(1, int(h * scale))

resized = cv2.resize(crop, (new_w, new_h), interpolation=cv2.INTER_AREA)
# Ensure sharp binary
resized = cv2.threshold(resized, 100, 255, cv2.THRESH_BINARY)[1]

start_x = (tw - new_w) // 2
start_y = (th - new_h) // 2
pad_box[start_y:start_y+new_h, start_x:start_x+new_w] = resized
return pad_box

def segment_digits(binary_mask: np.ndarray, min_area=35, min_height=12) -> List[dict]:
    """
    Segments candidate digit regions from a preprocessed binary mask.
    Filters noise, sorts bounding boxes from Left to Right,
    and returns a list of normalized digit items.
    """
    # Use connected components to cleanly separate digits
    num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
        binary_mask, connectivity=8
    )

    candidates = []
    for label in range(1, num_labels):
        area = stats[label, cv2.CC_STAT_AREA]
        w = stats[label, cv2.CC_STAT_WIDTH]
        h = stats[label, cv2.CC_STAT_HEIGHT]
        x = stats[label, cv2.CC_STAT_LEFT]
        y = stats[label, cv2.CC_STAT_TOP]

        # Filter out tiny noise specks or border artifacts
        if area < min_area or h < min_height or w < 3:
            continue

        comp_mask = np.uint8(labels == label) * 255
        crop = comp_mask[y:y+h, x:x+w]

        norm_roi = normalize_roi(crop, DEFAULT_TARGET_SIZE)

        candidates.append({
            "label": label,
            "bbox": (x, y, w, h),
            "area": area,
            "aspect_ratio": float(w) / max(1, h),
            "raw_crop": crop,
            "normalized_roi": norm_roi
        })

    # Crucial step: Sort from Left to Right by x-coordinate
    candidates.sort(key=lambda item: item["bbox"][0])

    return candidates

```

رابعاً: كود التعرف المستقل على الأرقام (digit_recognizer.py):

```

"""
digit_recognizer.py - Digit recognition module using normalized template matching.
Independent from operator recognition. Evaluates digits against 0..9 templates
and computes a unified confidence metric.

```

```

"""

import os
import cv2
import numpy as np
from segmentation import segment_digits, normalize_roi

DEFAULT_CONFIDENCE_THRESHOLD = 0.70

class DigitRecognizer:
    def __init__(self, templates_dir="templates/digits"):
        self.templates_dir = templates_dir
        self.templates = {}
        self.load_templates()

    def load_templates(self):
        """Loads all digit templates 0..9."""
        for d in range(10):
            path = os.path.join(self.templates_dir, f"{d}.png")
            if os.path.exists(path):
                tpl = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
                # Ensure binary
                _, tpl_bin = cv2.threshold(tpl, 127, 255, cv2.THRESH_BINARY)
                self.templates[str(d)] = tpl_bin
            else:
                raise FileNotFoundError(f"Template not found: {path}")

    def score_match(self, roi_bin: np.ndarray, template_bin: np.ndarray) -> float:
        """
        Computes a unified similarity score (0.0 to 1.0) combining:
        1. Normalized Cross-Correlation (TM_CCOEFF_NORMED)
        2. Binary Intersection-over-Union (IoU)
        """
        # 1. TM_CCOEFF_NORMED
        res = cv2.matchTemplate(roi_bin, template_bin, cv2.TM_CCOEFF_NORMED)
        corr = float(res[0][0])
        corr_score = max(0.0, corr)

        # 2. IoU
        intersection = np.sum((roi_bin > 0) & (template_bin > 0))
        union = np.sum((roi_bin > 0) | (template_bin > 0))
        iou_score = float(intersection) / max(1.0, float(union))

        # Unified confidence
        confidence = 0.6 * corr_score + 0.4 * iou_score
        return round(float(confidence), 4)

    def recognize_single_digit(self, norm_roi: np.ndarray) -> dict:
        """Matches a single normalized digit ROI against templates 0..9."""
        best_char = None
        best_conf = -1.0
        scores = {}

        for d_str, tpl in self.templates.items():
            conf = self.score_match(norm_roi, tpl)
            scores[d_str] = conf
            if conf > best_conf:
                best_conf = conf
                best_char = d_str

        return {
            "char": best_char,
            "confidence": best_conf,
            "all_scores": scores
        }

    def recognize_image(self, binary_mask: np.ndarray, min_conf=DEFAULT_CONFIDENCE_THRESHOLD)
-> dict:
        """
        Takes a binary mask (e.g. from preprocessing pipeline),
        segments digits from left to right, and recognizes each digit.
        Assembles multi-digit numbers (e.g., 9 and 2 -> 92).
        """
        segments = segment_digits(binary_mask)

```

```

        if not segments:
            return {
                "success": False,
                "value": None,
                "string": "",
                "confidence": 0.0,
                "digits": [],
                "error": "No digits segmented"
            }

        recognized_digits = []
        confidences = []

        for seg in segments:
            rec = self.recognize_single_digit(seg["normalized_roi"])
            recognized_digits.append({
                "char": rec["char"],
                "confidence": rec["confidence"],
                "bbox": seg["bbox"]
            })
            confidences.append(rec["confidence"])

        full_str = "".join([d["char"] for d in recognized_digits])

        # Unified overall confidence: weighted average with minimum penalty
        min_c = min(confidences)
        avg_c = sum(confidences) / len(confidences)
        overall_conf = round(0.7 * min_c + 0.3 * avg_c, 4)

        success = (overall_conf >= min_conf) and (len(full_str) > 0)

        try:
            val = int(full_str)
        except ValueError:
            val = None
            success = False

        return {
            "success": success,
            "value": val,
            "string": full_str,
            "confidence": overall_conf,
            "digits": recognized_digits,
            "error": None if success else f"Confidence {overall_conf:.2f} below threshold {min_conf:.2f}"
        }

```

خامساً: كود التعرف المستقل على العمليات (operator_recognizer.py):

```

"""
operator_recognizer.py - Operator recognition module (+, -, x, ÷).
Completely separated from digit recognition.
Combines simple geometric/shape analysis (Connected Components, aspect ratio,
stroke distribution) with normalized template matching for robust confirmation.
"""

import os
import cv2
import numpy as np
from edge_detection import find_connected_components, apply_canny
from segmentation import normalize_roi, DEFAULT_TARGET_SIZE

DEFAULT_CONFIDENCE_THRESHOLD = 0.70

class OperatorRecognizer:
    def __init__(self, templates_dir="templates/operators"):
        self.templates_dir = templates_dir
        self.templates = {}
        self.load_templates()

```

```

def load_templates(self):
    """Loads operator templates: add (+), sub (-), mul (x), div (÷)."""
    mapping = {
        "+": "add.png",
        "-": "sub.png",
        "x": "mul.png",
        "÷": "div.png"
    }
    for op, filename in mapping.items():
        path = os.path.join(self.templates_dir, filename)
        if os.path.exists(path):
            tpl = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
            _, tpl_bin = cv2.threshold(tpl, 127, 255, cv2.THRESH_BINARY)
            self.templates[op] = tpl_bin
        else:
            raise FileNotFoundError(f"Template operator not found: {path}")

def score_template(self, roi_bin: np.ndarray, template_bin: np.ndarray) -> float:
    """Unified template matching score (TM_CCOEFF_NORMED + IoU)."""
    res = cv2.matchTemplate(roi_bin, template_bin, cv2.TM_CCOEFF_NORMED)
    corr_score = max(0.0, float(res[0][0]))

    intersection = np.sum((roi_bin > 0) & (template_bin > 0))
    union = np.sum((roi_bin > 0) | (template_bin > 0))
    iou_score = float(intersection) / max(1.0, float(union))

    return 0.6 * corr_score + 0.4 * iou_score

def analyze_geometry(self, components: List[dict], binary_mask: np.ndarray) -> dict:
    """
    Simple geometric shape analysis using Connected Components:
    - 3 components aligned vertically (dot, bar, dot) -> '÷'
    - 1 component with W/H > 2.0 -> '-'
    - 1 component roughly square -> examine cross (+) vs diagonal (x)
    """
    num_comps = len(components)

    # 1. Division check: 3 vertically aligned components (or 2 if one dot merged/lost)
    if num_comps == 3:
        # Sort by y-coordinate (top to bottom)
        sorted_by_y = sorted(components, key=lambda c: c["bbox"][1])
        top_comp, mid_comp, bot_comp = sorted_by_y

        # Middle component should be horizontal bar (aspect ratio > 1.5)
        mid_ar = mid_comp["aspect_ratio"]
        if mid_ar > 1.4:
            return {"suggested": "÷", "confidence_bonus": 0.25, "reason": "3-component division (dot-bar-dot)"}
    elif num_comps == 2:
        # Sometimes 1 dot + 1 bar
        sorted_by_y = sorted(components, key=lambda c: c["bbox"][1])
        for comp in sorted_by_y:
            if comp["aspect_ratio"] > 1.6:
                return {"suggested": "÷", "confidence_bonus": 0.15, "reason": "2-component division"}

    # If 1 primary component (or after taking the largest):
    if num_comps >= 1:
        largest = max(components, key=lambda c: c["area"])
        ar = largest["aspect_ratio"]

        # Horizontal line check: Minus '-'
        if ar >= 2.0:
            return {"suggested": "-", "confidence_bonus": 0.25, "reason": "High aspect-ratio horizontal stroke (-)"}

        # Square-Like bounding box: Check + vs x
        if 0.7 <= ar <= 1.4:
            # Analyze central pixel distribution vs corners
            norm = normalize_roi(largest["crop"], (32, 32))
            h, w = norm.shape

            # In '+', middle cross (row 15-17, col 15-17) is filled, but 4 corners are
            empty

```

```

        # In 'x', corners have diagonals
        corner_sum = (
            np.sum(norm[2:8, 2:8] > 0) +
            np.sum(norm[2:8, 24:30] > 0) +
            np.sum(norm[24:30, 2:8] > 0) +
            np.sum(norm[24:30, 24:30] > 0)
        )

        # Cross arms at top-center, bottom-center, left-center, right-center
        plus_arms_sum = (
            np.sum(norm[2:8, 14:18] > 0) +
            np.sum(norm[24:30, 14:18] > 0) +
            np.sum(norm[14:18, 2:8] > 0) +
            np.sum(norm[14:18, 24:30] > 0)
        )

        if plus_arms_sum > corner_sum * 1.5:
            return {"suggested": "+", "confidence_bonus": 0.20, "reason": "Axis-
aligned cross (+)"}
        elif corner_sum > plus_arms_sum * 1.2:
            return {"suggested": "x", "confidence_bonus": 0.20, "reason": "Diagonal
strokes (x)"}

        return {"suggested": None, "confidence_bonus": 0.0, "reason": "Generic shape"}

    def recognize_image(self, binary_mask: np.ndarray, min_conf=DEFAULT_CONFIDENCE_THRESHOLD)
-> dict:
        """
        Recognizes operator from binary mask (+, -, x, ÷).
        Integrates Canny edge information, Connected Components, and Template Matching.
        """
        # Extract connected components (ignoring tiny noise < 15px)
        comps = find_connected_components(binary_mask, min_area=15)
        if not comps:
            return {
                "success": False,
                "symbol": None,
                "confidence": 0.0,
                "reason": "No components found",
                "all_scores": {}
            }

        # Step 1: Geometric structural analysis
        geom = self.analyze_geometry(comps, binary_mask)

        # Step 2: Extract unified ROI covering all valid operator components
        min_x = min(c["bbox"][0] for c in comps)
        min_y = min(c["bbox"][1] for c in comps)
        max_x = max(c["bbox"][0] + c["bbox"][2] for c in comps)
        max_y = max(c["bbox"][1] + c["bbox"][3] for c in comps)

        overall_crop = binary_mask[min_y:max_y, min_x:max_x]
        norm_roi = normalize_roi(overall_crop, DEFAULT_TARGET_SIZE)

        # Step 3: Template Matching across +, -, x, ÷
        scores = {}
        for op, tpl in self.templates.items():
            base_score = self.score_template(norm_roi, tpl)
            # Apply geometric bonus if geometry agrees
            if geom["suggested"] == op:
                base_score = min(1.0, base_score + geom["confidence_bonus"])
            scores[op] = round(float(base_score), 4)

        # Pick best match
        best_op = max(scores, key=scores.get)
        best_conf = scores[best_op]

        success = (best_conf >= min_conf)

        return {
            "success": success,
            "symbol": best_op,
            "confidence": best_conf,
            "bbox": (min_x, min_y, max_x - min_x, max_y - min_y),

```

```

        "geometry": geom,
        "all_scores": scores,
        "error": None if success else f"Confidence {best_conf:.2f} below threshold
{min_conf:.2f}"
    }

```

سادساً: كود المحرك الحسابي الآمن (calculator.py):

```

"""
calculator.py - Pure mathematical evaluation module.
Safe evaluation of arithmetic operations without using eval().
Handles division by zero gracefully and formats output cleanly.
"""

def add(a: float, b: float) -> float:
    return a + b

def subtract(a: float, b: float) -> float:
    return a - b

def multiply(a: float, b: float) -> float:
    return a * b

def divide(a: float, b: float) -> float:
    if b == 0:
        raise ZeroDivisionError("Error: Division by zero")
    return a / b

OPERATORS = {
    "+": add,
    "add": add,
    "-": subtract,
    "sub": subtract,
    "_": subtract,
    "x": multiply,
    "mul": multiply,
    "*": multiply,
    "X": multiply,
    "÷": divide,
    "div": divide,
    "/": divide
}

DISPLAY_SYMBOLS = {
    "+": "+",
    "add": "+",
    "-": "-",
    "sub": "-",
    "_": "-",
    "x": "x",
    "mul": "x",
    "*": "x",
    "X": "x",
    "÷": "÷",
    "div": "÷",
    "/": "÷"
}

def format_number(val: float) -> str:
    """Formats a number cleanly: no trailing .0 for integers, up to 4 decimal places for floats."""
    if abs(val - round(val)) < 1e-9:
        return str(int(round(val)))
    formatted = f"{val:.4f}".rstrip('0').rstrip('.')
    return formatted

def calculate_expression(num1: int | float, op_symbol: str, num2: int | float):
    """

```

```

Safely calculates num1 [op] num2 without using eval().
Returns a dict with status, result_str, formatted_expr, error_msg.
"""
op_key = str(op_symbol).strip()
if op_key not in OPERATORS:
    return {
        "success": False,
        "error": f"Invalid or unrecognized operator: '{op_symbol}'",
        "expression": f"{num1} {op_symbol} {num2}",
        "result_str": None,
        "result_val": None
    }

func = OPERATORS[op_key]
disp_op = DISPLAY_SYMBOLS.get(op_key, op_key)
expr_str = f"{format_number(num1)} {disp_op} {format_number(num2)}"

try:
    res = func(float(num1), float(num2))
    res_str = format_number(res)
    return {
        "success": True,
        "error": None,
        "expression": expr_str,
        "result_str": res_str,
        "result_val": res,
        "full_equation": f"{expr_str} = {res_str}"
    }
except ZeroDivisionError as zde:
    return {
        "success": False,
        "error": str(zde),
        "expression": expr_str,
        "result_str": "Undefined (ZeroDivision)",
        "result_val": None,
        "full_equation": f"{expr_str} = Undefined (Division by zero)"
    }
except Exception as e:
    return {
        "success": False,
        "error": str(e),
        "expression": expr_str,
        "result_str": "Error",
        "result_val": None,
        "full_equation": f"{expr_str} = Error"
    }

```

سابعاً: كود إدارة المعالجة وإعادة المحاولة (image_processor.py):

```

"""
image_processor.py - Unified orchestrator for DIP pipelines, fallback retries,
and full expression processing.
Executes Pipeline 1 -> Pipeline 2 -> Pipeline 3 fallback strategy
to pick the highest confidence result.
"""

import cv2
import numpy as np
from preprocessing import run_pipeline
from edge_detection import apply_canny, clean_edges_with_components
from digit_recognizer import DigitRecognizer, DEFAULT_CONFIDENCE_THRESHOLD
from operator_recognizer import OperatorRecognizer
from calculator import calculate_expression

class ImageProcessor:
    def __init__(self, templates_digits_dir="templates/digits",
                 templates_ops_dir="templates/operators"):
        self.digit_recognizer = DigitRecognizer(templates_digits_dir)
        self.operator_recognizer = OperatorRecognizer(templates_ops_dir)
        self.pipeline_names = ["pipeline_1", "pipeline_2", "pipeline_3"]
        self.target_conf_threshold = DEFAULT_CONFIDENCE_THRESHOLD

```

```

def process_number(self, image: np.ndarray, min_conf=None) -> dict:
    """
    Processes a number image through the DIP pipeline with fallback retries.
    Tries Pipeline 1 -> 2 -> 3 if confidence is below threshold,
    and selects the candidate with the highest confidence.
    """
    threshold = min_conf if min_conf is not None else self.target_conf_threshold
    best_result = None
    highest_conf = -1.0

    for p_name in self.pipeline_names:
        prep_data = run_pipeline(image, pipeline_name=p_name)
        edges = apply_canny(prepare_data["denoised"])
        clean_edges = clean_edges_with_components(edges, prep_data["morph"])

        rec_res = self.digit_recognizer.recognize_image(prepare_data["morph"],
min_conf=threshold)

        candidate = {
            "type": "number",
            "pipeline_used": p_name,
            "success": rec_res["success"],
            "value": rec_res["value"],
            "string": rec_res["string"],
            "confidence": rec_res["confidence"],
            "digits": rec_res["digits"],
            "stages": {
                "original": prep_data["original"],
                "gray": prep_data["gray"],
                "denoised": prep_data["denoised"],
                "binary": prep_data["binary"],
                "morph": prep_data["morph"],
                "canny": edges,
                "clean_edges": clean_edges
            }
        }

        if rec_res["confidence"] > highest_conf:
            highest_conf = rec_res["confidence"]
            best_result = candidate

        # If we achieved very high confidence, no need to execute remaining pipelines
        if rec_res["success"] and rec_res["confidence"] >= 0.88:
            break

    return best_result

def process_operator(self, image: np.ndarray, min_conf=None) -> dict:
    """
    Processes an operator image through the DIP pipeline with fallback retries.
    Selects the result with the highest confidence across pipelines.
    """
    threshold = min_conf if min_conf is not None else self.target_conf_threshold
    best_result = None
    highest_conf = -1.0

    for p_name in self.pipeline_names:
        prep_data = run_pipeline(image, pipeline_name=p_name)
        edges = apply_canny(prepare_data["denoised"])
        clean_edges = clean_edges_with_components(edges, prep_data["morph"])

        rec_res = self.operator_recognizer.recognize_image(prepare_data["morph"],
min_conf=threshold)

        candidate = {
            "type": "operator",
            "pipeline_used": p_name,
            "success": rec_res["success"],
            "symbol": rec_res["symbol"],
            "confidence": rec_res["confidence"],
            "bbox": rec_res.get("bbox"),
            "geometry": rec_res.get("geometry"),
            "all_scores": rec_res.get("all_scores"),

```

```

        "stages": {
            "original": prep_data["original"],
            "gray": prep_data["gray"],
            "denoised": prep_data["denoised"],
            "binary": prep_data["binary"],
            "morph": prep_data["morph"],
            "canny": edges,
            "clean_edges": clean_edges
        }
    }

    if rec_res["confidence"] > highest_conf:
        highest_conf = rec_res["confidence"]
        best_result = candidate

    if rec_res["success"] and rec_res["confidence"] >= 0.90:
        break

    return best_result

def process_expression(self, img_num1: np.ndarray, img_op: np.ndarray, img_num2:
np.ndarray) -> dict:
    """
    Orchestrates full 3-image calculation:
    1. Recognizes number 1
    2. Recognizes operator
    3. Recognizes number 2
    4. Calculates mathematical expression without eval()
    """
    res_num1 = self.process_number(img_num1)
    res_op = self.process_operator(img_op)
    res_num2 = self.process_number(img_num2)

    # Check if all 3 parts were recognized
    all_recognized = (
        res_num1["value"] is not None and
        res_op["symbol"] is not None and
        res_num2["value"] is not None
    )

    calc_res = None
    if all_recognized:
        calc_res = calculate_expression(
            res_num1["value"],
            res_op["symbol"],
            res_num2["value"]
        )
    else:
        missing = []
        if res_num1["value"] is None: missing.append("First Number")
        if res_op["symbol"] is None: missing.append("Operator")
        if res_num2["value"] is None: missing.append("Second Number")

        calc_res = {
            "success": False,
            "error": f"Recognition incomplete for: {'', '.join(missing)}",
            "expression": f"{res_num1['string'] or '?'} {res_op['symbol'] or '?'} {res_num2['string'] or '?'}",
            "result_str": "N/A",
            "result_val": None,
            "full_equation": "Incomplete Expression"
        }

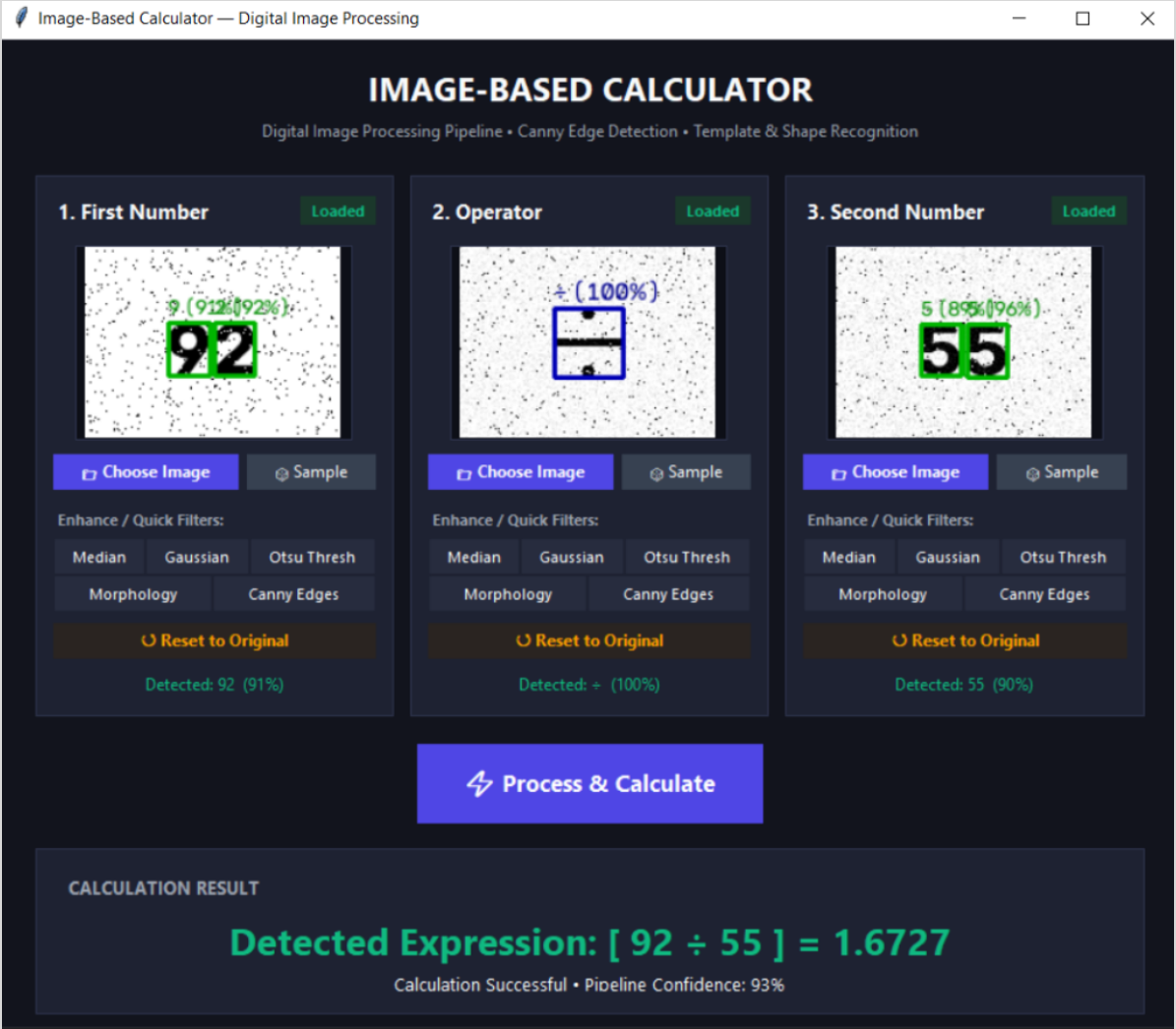
    overall_confidence = round(
        float(res_num1["confidence"] * 0.35 + res_op["confidence"] * 0.30 +
        res_num2["confidence"] * 0.35),
        4
    )

    return {
        "num1": res_num1,
        "op": res_op,
        "num2": res_num2,
        "calculation": calc_res,
    }

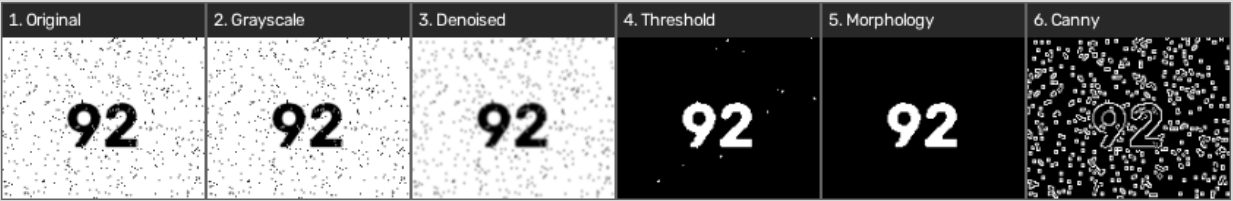
```

```
}
    "overall_confidence": overall_confidence
}
```

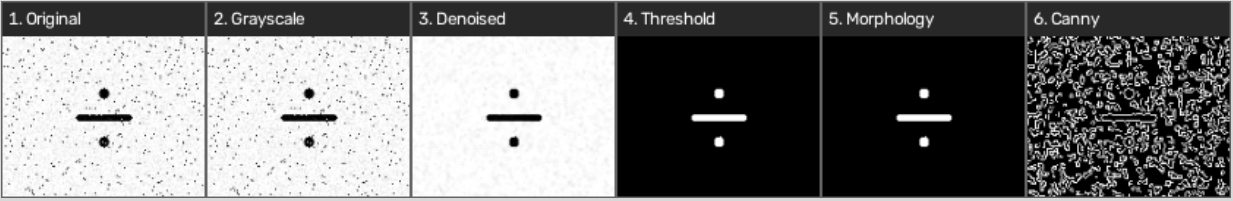
6. صور التنفيذ ونتائج الاختبار (Execution Screenshots & Results):



(Original -> Gray -> Denoised -> Threshold -> Morph -> Canny)



(Original -> Gray -> Denoised -> Threshold -> Morph -> Canny)



(Original -> Gray -> Denoised -> Threshold -> Morph -> Canny)

1. Original	2. Grayscale	3. Denoised	4. Threshold	5. Morphology	6. Canny
					